

PLAY · 접속 방법

<http://choseongchan.kro.kr/>

PC 브라우저(Chrome·Edge 권장)로 접속 → 이름 입력 → 바로 전투 시작. 별도 설치·회원가입 없음.

▶ 플레이 영상 [youtube.com/watch?v=6L8eNiC1DhA](https://www.youtube.com/watch?v=6L8eNiC1DhA)

개인 PC에서 서버를 직접 운영하고 있어 접속되지 않을 수 있습니다. 접속이 안 되면 아래 연락처로 연락 부탁드립니다.

aa01053547426@gmail.com

연락처 010-5354-7426

GAME PORTFOLIO

escape_from_blockov

WebGL 2.5D 탑다운 PvP 슈팅 — 자체 IOCP 서버 엔진 위에 만든

대규모 동시 접속 게임 서버와 Unity WebGL 클라이언트

C++20 / IOCP

Lock-free · TLS Object Pool

Unity 6 WebGL (C#)

WebSocket ↔ TCP Gateway (Node.js)

지연 보상

섹터 시야

C++ IOCP 더미 클라이언트

목차

[01 프로젝트 개요 · 개발 분담](#)

[02 시스템 아키텍처](#)

[03 서버 엔진 NetLib](#)

[04 게임 서버 · 방\(Content\) 구조](#)

[05 네트워크 프로토콜](#)

[06 시야 · 섹터 / 트래픽 추정](#)

[07 이동 동기화 · 서버 검증](#)

[08 사격 판정 · 지연 보상](#)

[09 시각 동기화](#)

[10 치트 대응 · 연결 종료 정책](#)

[11 맵 파이프라인 · 파괴 가능 얀페물](#)

[12 게임 시스템](#)

[13 클라이언트 아키텍처](#)

[14 테스트 · 부하 측정 · 운영](#)

[15 트러블슈팅 사례](#)

[16 확장 계획 · 회고](#)

기준 문서: 게임 명세서 v0.15 (2026-10-01), ContentServer 라이브러리 가이드, 맵 섹터 규격.

프로젝트 개요

로비 없이 이름만 입력하면 바로 전장에 들어가는 브라우저 PvP 슈팅. 1.5km 맵에서 적을 처치해 점수를 흡수하고, 죽으면 모든 것을 잃는 slither.io(웹 지렁이 게임)식 구조이다.

게임 한 판의 흐름

접속 → 스폰(주변 인원이 가장 적은 섹터) → 탐색·이동·사격·아이템 획득 → 처치 시 상대 점수의 50% 흡수 → 사망 → 결과창 → 타이틀. 계정·DB 없이 서버 메모리만으로 동작하며, 방 안 상위 3명이 실시간 랭킹에 표시된다.

핵심 기능

- WASD 이동, Shift 달리기, Space 구르기(서버가 도착점 계산)
- 권총 · 샷건(산탄 5발) · 저격총, 내구도, 붕대 회복
- 에어드랍(인원 밀집 지점에 투하, 30초 전 예고) · 사망 가방 루팅
- BMP로 그리는 엄폐물 맵: 벽 / 낮은 엄폐물 / 파괴 가능 엄폐물
- 전체 맵(M) · 구역 번호 1~100 · 화면 밖 에어드랍 방향 화살표
- 거리 감쇠·좌우 팬을 적용한 합성 효과음



WebGL 플레이 화면 — 상단 접속 인원, 우상단 TOP3, 하단 HP·점수·슬롯 바·조작 안내

항목	내용
장르/ 플랫폼	2.5D 탐다운 PvP 슈팅 · Unity WebGL
목표 규모	방당 동시 300인 플레이 / 방 정원 300 × 방 4개 (확장 가능)
맵	1500 × 1500 m, 50m 섹터 30×30
개발 형태	1인 개발 · 3일 — 기존에 직접 만든 서버 엔진(NetLib-Utils)을 재사용하고, 그 위의 게임 콘텐츠·클라·도구는 AI 활용(01 · 개발 분담)

기술 스택과 코드 규모

서버 엔진부터 클라이언트·도구까지 한 저장소에서 관리한다.

영역	기술	규모 · 비교
서버 엔진 NetLib	C++20, Win32 IOCP, lock-free 큐/스택, TLS 오브젝트 풀	NetLib 3.5k줄 + Utils 2.3k줄. 세션 I/O 워커 풀 + Content 틱 루프
게임 서버	NetLib 위 EntryContent / BattleContent	약 3.6k줄(본체) + Linux 스텝 테스트 하네스
클라이언트	Unity 6000.x, URP, C#, jslib(WebSocket·입력 오버레이)	런타임 스크립트 32개 4.7k줄, 에디터 도구 0.5k줄
게이트웨이	Node.js 18 내장 모듈만 사용(외부 패키지 없음)	WebSocket 핸드셰이크·프레임 파서 직접 구현, 160줄
더미 클라이언트	C++ IOCP 콘솔, 서버 프로토콜·맵 코드 공유	2.4k줄. 교전 AI, 최대 2만 더미
도구	Python(numpy/scipy 맵 생성, 효과음 합성), Node 봇 테스트	start_server.bat 원클릭 실행, 크래시 덤프, 대시보드

줄 수는 저장소의 .cpp/.h/.cs/.js 기준 대략치.

직접 구현한 부분과 AI를 사용한 부분

서버 라이브러리(NetLib), 콘텐츠 서버 구조, 그리고 그 유틸리티 도구들은 직접 설계·구현했다. AI(Claude)는 그 라이브러리 위에 올라가는 게임 콘텐츠와 주변 도구를 만드는 데에만 사용했다.

AI 사용 (Claude) — 개발자의 기획·요구 사항·결정에 따라 구현

게임 서버 콘텐츠

Unity WebGL 클라이언트

게이트웨이 · 더미 · 테스트 · 맵 도구

명세 · 분석 문서

▲ 라이브러리의 Content · 송신 API · 세션 모델을 그대로 사용 ▲

직접 구현 (본인) — 이 프로젝트 이전부터 작성한 코드

NetLib (IOCP 서버 · Content 구조 · 세션 · 패킷) · Utils (TLS 풀 · lock-free 자료구조 · 로거 · 프로파일러 · 모니터 · 덤프) · 예제 Echo 서버

그림 1-1. 계층별 분담 — 서버의 아키텍처·스레딩 모델·세션 수명·송신 소유권 규칙은 모두 직접 만든 라이브러리에서 나온다

직접 구현 — server/ContentEchoServer/

IOCP 서버 라이브러리 (NetLib/)

NetLib_Server	IOCP, Accept/Worker/Monitor 스레드, 세션 배열·인덱스 풀, 송신 API 4종, 세션 해제, Content 등록·이동
NetLib_Content	콘텐츠 서버 구조: IOCP 위 고정 틱 Update 루프, ENTER/LEAVE/RELEASE 메시지 큐, Move_Content 중 패킷 순서 보존
Session	recv·콘텐츠 수신 링버퍼, lock-free 송신 큐, IOCount + Release 플러그, 64bit 세션 핸들
Packet / SerializeBuffer	직렬화, TLS 풀 기반 패킷, refCount, 헤더 예약
Protocol · Define · Helper	NetHeader 5B, 경량 암호화 옵션, overlapped 타입 구분
NetLib_Client	서버 간 통신용 IOCP 클라이언트

유틸리티 (Utils/)

TLXObjectPool(스레드별 청크 + 공용 저장소) · LockFreeQueue/Stack/Pool(카운터 태깅 ABA 방지) · ObjectPool · RingBuffer · SimpleEncoder · Logger · Profiler · Parser · SystemMonitor(PDH) · CrashDump

예제 콘텐츠 서버

ContentServer · AuthContent · EchoContent · MonitorClient — 인증 → 에코 Content 흐름과 모니터 서버 연동. 게임 서버는 이 구조(서버 클래스 → 입장 Content → 게임 Content)를 그대로 따른다.

AI 사용 (Claude)

영역	내용
게임 서버 콘텐츠	server/GameServer/ — 입장·방 배정, 전투 방(이동·사격 검증, 퇴각기 피격 판정, 점수·랭킹, 아이템·가방·에어드랍, 파괴 가능 엄폐물), 엄폐물 맵, 설정
게임 클라이언트	client/escape_from_blockov/ — 통신 모듈, 게임 로직, HUD, 맵 임포트 도구, 효과음·이펙트
주변 도구	WS↔TCP 게이트웨이, 더미 클라이언트, 테스트 봇·리눅스 스텝, 예시 맵 생성기, 실행 배치, WebGL 웹서버
문서	게임 명세(GAME_SPEC.md), 라이브러리 분석 문서(기존 소스를 읽고 정리한 사용 가이드)

일하는 방식

- 기획·요구 사항·결정은 개발자, AI는 그에 따라 명세 문서를 갱신하고 코드를 구현.
- 명세서(v0.1 → v0.15)를 단일 기준으로 두고, 결과는 더미 부하 테스트로 검증.
- 라이브러리에 대한 AI의 수정은 금지.(제공되는 인터페이스만 사용 가능)

요약 핵심 서버 기술 — IOCP 네트워크 라이브러리, 콘텐츠 서버 구조, lock-free 자료구조·오브젝트 풀 등을 직접 구현했다. AI는 그 위의 게임 콘텐츠(게임 서버 로직, Unity 클라이언트)와 테스트·운영 도구, 문서 작성에 사용했다.

시스템 아키텍처

브라우저는 TCP를 쓸 수 없으므로 WebSocket으로 들어오고, 게이트웨이가 바이트를 그대로 TCP로 중계한다. 게임 서버는 사람과 더미를 구분하지 않는다.

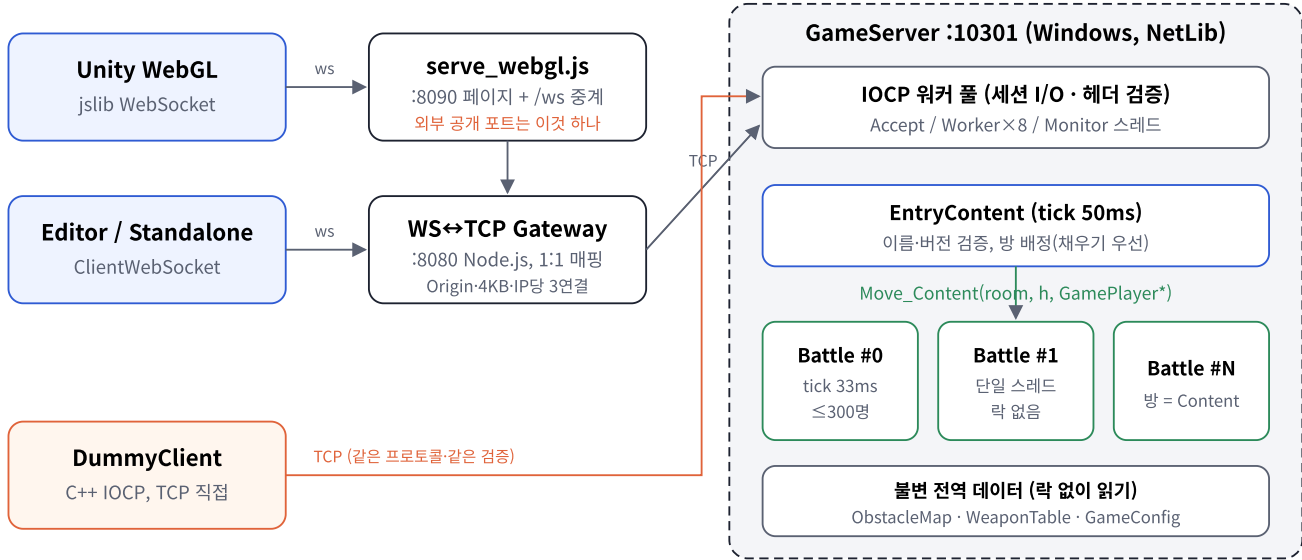


그림 2-1. 현재 배포 구성. 브라우저 → :8090(/ws) → 게이트웨이 :8080 → 게임 서버 :10301

WS 메시지 ↔ 패킷 규약

- 클라→서버: WS 메시지 1개 = NetHeader 패킷 **정확히 1개**
- 서버→클라: WS 메시지 1개 = 패킷 **1개 이상**(경계 비보장)
- 수신측은 메시지 경계를 믿지 않고 **바이트 스트림으로 재조립**

왜 웹소켓 ↔ TCP 로 분리했는가?

IOCP 엔진에 HTTP 업그레이드·WS 프레이밍을 바로 넣으면 세션 상태와 recv 흐름을 함께 고쳐야 한다. 먼저 브라우저 경로를 열고, 엔진 쪽 WS 지원은 같은 규약을 유지하는 후속 단계(16장)로 분리했다. 덕분에 서버는 TCP 더미 클라이언트로 그대로 부하 테스트할 수 있다.

프로세스 · 포트 구성

프로세스	포트	역할
serve_webgl.js	80 (외부 공개)	WebGL 빌드 정적 서빙(Builds/WebGL 밖은 403, GET/HEAD만) + /ws WebSocket 중계
gateway/index.js	8080 (127.0.0.1)	WS 연결 1개 ↔ TCP 연결 1개, 바이너리 프레임만 허용, 단편화 재조립, ping/pong/close 처리
GameServer.exe	10301 (내부)	EntryContent 1 + BattleContent N, 1초 주기 콘솔 대시보드

start_server.bat [build] [web] 한 번으로 세 프로세스를 띄우고(필요 시 MSBuild 빌드), stop_server.bat 로 일괄 종료한다.

서버 엔진 NetLib

세션 I/O는 IOCP 워커 풀이, 게임 로직은 고정 틱으로 도는 **Content**가 맡는다. Content 하나는 동시에 한 워커에서만 실행되므로 방 내부 상태는 락 없이 다룬다.
자세한 설명은 <조성찬_IOCP네트워크라이브러리구축과활용>에서 다룬다.

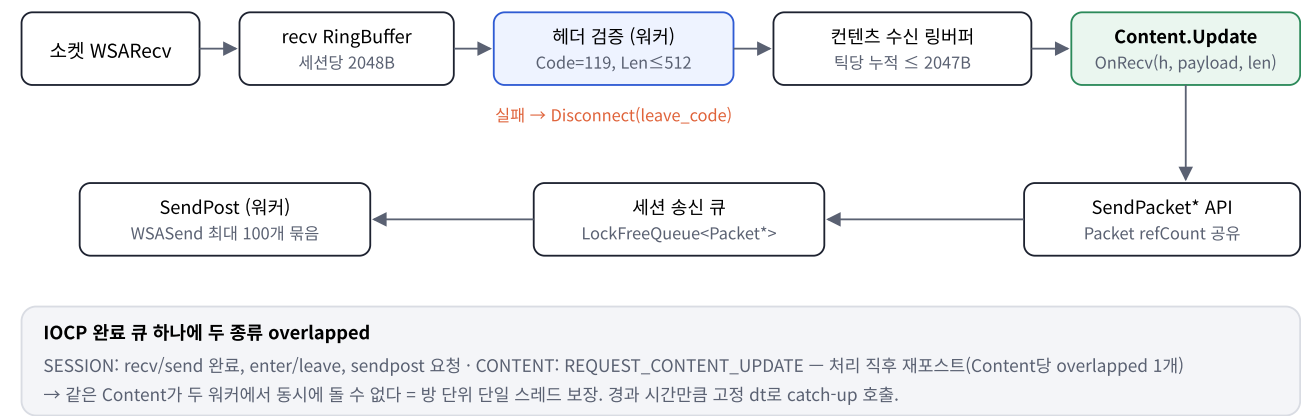


그림 3-1. 수신 → 컨텐츠 → 송신 데이터 흐름과 스레드 모델

세션 수명: IOCount + 세대 핸들

- sessionHandle = (index<<48) | id48 — 재사용된 슬롯에 대한 늛은 접근을 핸들 불일치로 걸러낸다.
- 16bit IOCount = 상위 8bit Release 플래그(0xEE) + 하위 8bit 참조 수. 증가 시 플래그가 서 있으면 즉시 손을 뗀다.
- Disconnect는 CancelIoEx만 걸고, 진행 중 I/O가 모두 끝나 IOCount가 0이 될 때 해제 — 해제와 I/O 완료의 경합 제거.
- Content에 속한 세션은 그 Content가 RELEASE 메시지를 처리하기 전까지 해제되지 않는다.

Content 이동과 패킷 순서 보존

Move_Content 중에는 IsMoving 플래그로 새 Content가 수신 버퍼를 읽지 않는다. 이전 Content가 LEAVE를 처리한 뒤 새 Content가 ENTER부터 이어서 읽으므로, Entry→Battle 전환 순간에 도착한 패킷도 순서가 유지된다.

송신 인터페이스 소유권 규칙

API	패킷 / 해제	사용처
SendPacket	Alloc, 호출자 Free	어디서나
Multicast	Alloc 1회 → refCount n 공유, 호출자 Free	3×3·방 방송
SendPacketFast	NetAlloc(헤더 예약), 라이브러리 해제	복사 없음
...WithoutIOCount	NetAlloc, 라이브러리 해제	소속 Content 콜백 안에서만

게임 서버 규칙: 본인 응답은 NetAlloc + FastWithoutIOCount (인터락 연산 생략), 다수 대상은 Alloc + Multicast. 핸들 배열은 방 정원 크기로 미리 확보해 틱 중 할당이 없다.

메모리 · 자료구조

- TLS 오브젝트 풀: 스레드별 청크(사용/반납) + 공용 청크 저장소. Packet·큐 노드에 사용.
- 카운터 태깅 lock-free 큐/스택으로 ABA 방지.
- 워커가 하는 일은 헤더 검증과 버퍼 복사뿐 — 게임 로직은 모두 Content 틱 안.

설계 판단 방(룸)을 Content 인스턴스 하나로 두면 "방 = 단일 스레드 = 락 없는 게임 상태"가 된다. 방끼리는 병렬이고, 방 사이에 공유하는 것은 불변 데이터(맵·무기표)와 원자 카운터(방 예약 수)뿐이다. 대가는 Content가 바쁜 루프라는 점 — 방 수 + 1 ≤ 워커 수 - 2로 세션 I/O 워커를 남겨 둔다.

게임 서버 · 방(Content) 구조

NetLib의 예제 서버(Auth/Echo)를 실제 게임 서버로 확장했다. 엔트리에서 이름·버전을 받고, 방(BattleContent)으로 넘긴 뒤에는 그 방이 플레이어의 모든 상태를 소유한다.

클래스	기반	역할
GameServer	NetLib_Server	설정 로드, Entry 1 + Battle N 생성·등록, OnClientJoin → Move_Content(entry)
EntryContent	Content, 50ms	CS_ENTER_GAME 10초 대기, 버전·이름 정규화, 방 선택, GamePlayer 생성 후 방으로 이동
BattleContent	Content, 33ms	방 1개: 플레이어·섹터·전투 판정·아이템·컨테이너·에어드랍·엄폐물 상태·랭킹 전부

방 배정: 예약 카운터로 채우기 우선

GetSessionCount() 는 방이 ENTER 메시지를 처리해야 늘어나므로, 동시에 몰린 입장 요청을 판단하는 데 쓸 수 없다. 방마다 std::atomic<int> reserved 를 두고 번호 순으로 fetch_add → 정원 초과면 되돌리고 다음 방. 전부 가득이면 SERVER_FULL 후 1초 뒤 종료, OnRelease 에서 reserved-- .

BattleContent 한 틱 (33ms)

- ENTER/LEAVE/RELEASE 메시지 처리
- 소속 세션의 수신 패킷 전부 OnRecv — 타입별 정확한 길이 검증, 불일치 시 SC_KICK(INVALID_PACKET)
- OnUpdate : 이동 예산·사격 토큰 충전, 붕대 타이머, 가방 만료, 에어드랍 예고·투하, 엄폐물 재생 (0.5초), 정지 플레이어 이력 보충(200ms)
- 틱 끝 병합 방송: 랭킹이 바뀌었으면 SC_RANKING_TOP3 1회, 참여 인원 변화는 200ms 간격으로 SC_PLAYER_COUNT

설정 주도 (game_config.txt)

포트·워커 수·방 수/정원·틱·시야 반경·타임아웃·이동/구르기/붕대/에어드랍/엄폐물 재생 등 40여 개 값을 설정 파일에서 읽는다. 무기 수치는 weapons.txt (15열)에서 읽어 입장 시 SC_WEAPON_DEFS 로 내려보내므로 밸런스 조정에 클라 재빌드가 필요 없다.

무기	데미지	사거리	간격	특징
Pistol	20	32	250ms	무한, 흔들림 ±3°
Shotgun	20×5	25	1000ms	퍼짐 12°, 내구도 40
Sniper	60	45	2000ms	탄속 200, 내구도 30

작업 폴더 자동 맞춤 · 크래시 덤프

설정 파일이 없으면 실행 파일 위쪽 폴더를 찾아 작업 폴더를 옮긴다(exe 직접 실행 시 상대 경로 문제 해결). 처리되지 않은 SEH·CRT 오류·abort·std::terminate 를 잡아 dumps/ 에 전체 메모리 덤프를 프로세스당 1회 남기고, 덤프 직전 게임 로그에 한 줄을 남긴다.

네트워크 프로토콜

고정 5바이트 헤더 + 리틀 엔디안 packed 구조체. 서버 C++ 헤더(GameProtocol.h)와 클라 C#(NetProtocol.cs), 더미 C++이 같은 정의를 따른다. 현재 v7.

```
| Code(1)=0x77 | Len(2) ≤ 512 | RandKey(1)=0 | CheckSum(1)=0 | Payload: WORD Type + 본문 |
- #pragma pack(1), 리틀 엔디안, 문자열은 WCHAR Name[12] 고정 24B (UTF-16LE)
- 방향은 정규화 2D 벡터(float×2): 길이 0.9~1.1 밖이면 거부, 통과 시 재정규화
- 시각은 UINT32 서버 ms (49일 wrap 허용, 비교는 부호 있는 차이)
- PlayerID는 방 내 1부터 증가. 내부 sessionHandle은 클라에 노출하지 않음
```

패킷 구성 (C→S 11종 · S→C 25종)

분류	주요 패킷
입장	CS_ENTER_GAME / SC_ENTER_GAME(65B, 맵 해시·서버 시각 포함), SC_WEAPON_DEFS
이동	CS_MOVE(24B) / SC_MOVE, SC_POSITION_CORRECT, CS_ROLL / SC_ROLL
전투	CS_FIRE(29B), CS_HIT_REPORT(3+17n, n≤29) / SC_FIRE, SC_DAMAGE, SC_PLAYER_DIE
시야	SC_CREATE_CHARACTERS(54B×9), SC_DELETE_CHARACTERS
아이템	CS_SWITCH_WEAPON, CS_USE_BANDAGE, CS_OPEN_CONTAINER, CS_TAKE_ITEM / SC_INVENTORY, SC_CONTAINER_*
월드	SC_AIRDROP, SC_AIRDROP_FORECAST, SC_COVER_HP, SC_COVER_STATE
메타	CS_PING/SC_PONG, CS_HEARTBEAT, SC_SCORE, SC_RANKING_TOP3, SC_PLAYER_COUNT, SC_KICK

입장 시퀀스 (순서 고정)

ENTER_GAME → PLAYER_COUNT → WEAPON_DEFS → INVENTORY → CREATE_CHARACTERS → CONTAINER_CREATE → AIRDROP(기존) → AIRDROP_FORECAST → COVER_STATE(파괴 목록) → RANKING_TOP3

클라는 SC_ENTER_GAME(OK) 이후 패킷을 보류했다가 전투 씬의 GameController가 준비되면 풀어 준다 — 씬 로딩 중 도착한 패킷의 순서와 유실을 막는다.

버전 관리 프로토콜을 바꿀 때마다 GAME_PROTOCOL_VERSION · NetConst.ProtocolVersion · Node 테스트 봇의 길이 상수를 함께 올린다. 버전이 다르면 VERSION_MISMATCH 로 입장을 거부해, 캐시된 옛 WebGL 빌드가 새 서버에 섞여 들어오는 일을 막는다. 같은 원리로 맵도 해시로 대조한다(11장).

시야 · 섹터 / 트래픽 추정

1500m 맵을 50m 섹터 30×30으로 나누고, 플레이어는 자기 섹터와 인접 8섹터(3×3) 안의 정보만 주고받는다. 브로드캐스트 비용을 "방 인원"이 아니라 "주변 인원"에 비례하게 만드는 장치다.



그림 6-1. 섹터 경계를 넘을 때의 3×3 diff — 빠진 열과 새로 들어온 열만 처리한다

규칙

- $sx = \text{floor}(x/50)$, $sy = \text{floor}(z/50)$, 인덱스 $sy*30+sx$ — 서버 MapConst / SectorMap.h 와 클라 SectorGrid.cs 가 같은 식. (서버와 클라의 섹터 단위가 같다.)
- 살아있는 플레이어는 정확히 한 섹터에 등록. 사망·퇴장 시 즉시 제거.
- 브로드캐스트 범위: 이동·사격·피해·HP·구르기 = 3×3. 방 전체 = 랭킹·접속 인원·에어드랍·엄페물 파괴/재생.
- SC_DAMAGE 를 피해자 3×3에 보내면 사수도 자동으로 포함된다.

사거리와 시야의 정합

플레이어가 섹터 경계에 서 있으면 한 방향으로 보장되는 시야는 섹터 하나, 즉 **50m**다. 무기 사거리가 이보다 길면 "서버가 보여주지 않은 적을 맞추는" 상황이 생기므로, 서버가 무기표를 읽을 때 사거리를 $\text{섹터 크기} \times \text{view_sector_radius}(50)$ 로 자른다.

트레이드오프 섹터를 64m·100×100에서 50m·30×30으로 바꾸며(v0.7) 월드를 1.5km로 줄이고 사거리 상한을 맞췄다. 엄페물은 이동·총알만 막고 시야(정보 송신)는 막지 않는다 — 벽 뒤 적의 정보가 클라에 있다는 점은 열린 이슈로 남겨 두었다.

트래픽 추정 (방 1개, 전원이 한 3×3에 밀집)

방 인원	SC_MOVE (10Hz, 31B)	SC_FIRE (4Hz, 33B)	합계
50	≈ 24.5k pkt/s · 760 KB/s	9.8k pkt/s · 325 KB/s	≈ 1.1 MB/s
100	≈ 99k pkt/s · 3.1 MB/s	39.6k pkt/s · 1.3 MB/s	≈ 4.4 MB/s

밀집 시 비용은 시야 내 인원의 제곱. 그래서 맵을 넓게 두고 교전을 드물게 만드는 것이 기획 의도이자 확장 전략이다. 클라는 이동 중이거나 조준이 5° 이상 바뀔 때만 100ms 간격(속도 변화 시 즉시, 최소 50ms)으로 SC_MOVE 를 보내 정지 상태의 트래픽을 0으로 만든다.

이동 동기화 · 서버 검증

반응성을 위해 이동은 클라가 먼저 하고(예측), 서버는 결과가 물리적으로 가능한지만 검증한다. 위반이면 서버 위치로 되돌리고(`SC_POSITION_CORRECT`) 위반 횟수를 센다.

CS_MOVE 검증 순서

1. NaN/Inf, 맵 경계 `[2, 1498]` 밖 → 클램프 후 보정
2. 보고 속도 `|Vel| ≤ MaxSpeed × 1.1`, 조준각 유한값
3. **이동 예측**: 매 틱 `budget += MaxSpeed·dt·1.2`, 상한 1초분. 이동 거리 `d ≤ budget + 0.5` 면 통과 후 차감
4. **엠펙물**: 이전→새 위치 선분 DDA(`SegmentBlocked`, 이동 모드) + 새 위치 원(`CircleBlocked`, 반지름 0.4)
5. 통과 → 위치·이력 갱신, 섹터 변경 처리, 3×3에 `SC_MOVE`

왜 이동 예측인가? 패킷 간 시간으로 속도를 재면 네트워크 지터로 뭉쳐 도착한 패킷이 순간 과속처럼 보인다. 예측 방식은 짧은 뭉침은 허용하면서 **평균 속도**만 제한하므로, 정상 플레이에서 보정이 거의 나지 않는다(더미 300명 48시간 테스트 보정 0회).

구르기: 도착점은 서버가 계산

`CS_ROLL{시작 위치, 방향}` → 쿨타임(3초)·시작 위치(서버 위치 3m 이내)·방향 검증 → 0.25m 단위로 전진하며 첫 막힌 칸 앞에서 멈추는 **도착점을 서버가 직접 계산**해 확정한다. 클라도 같은 계산을 하므로 평소엔 일치한다.

- 위치 이력에 시작(now)·도착(now+250ms) 두 점을 기록 → 구르는 중 피격 판정도 보간된다.
- 구르는 0.25초 동안의 CS_MOVE는 무시하되 끝나기 100ms 전부터는 도착 편차로 인정.
- 구르기 후 첫 이동이 `1m + 경과×최고속도` 보다 멀면 보정하되 위반으로 세지 않는다.

클라 측 예측과 충돌

클라도 같은 BMP 격자(`obstacle_map.bytes`)로 0.25m 단위 충돌을 검사하고, 막히면 축별로 미끄러진다. 서버와 같은 데이터·같은 규칙을 쓰는 것이 보정 0의 전제이며, 이를 **맵 해시**로 강제한다(11장).

원격 캐릭터 보간

스냅샷 버퍼를 `EstServerNow - 100ms` 시점으로 보간하고, 스냅샷이 모자라면 속도로 최대 200ms 외삽한다.

사격 판정 · 지연 보상

"내 화면에서 맞았으면 맞은 것"을 목표로, 사수 클라가 판정하고 서버가 과거 위치로 되감아 검증하는 방식을 택했다. HP 감소는 서버의 SC_DAMAGE 로만 반영한다.

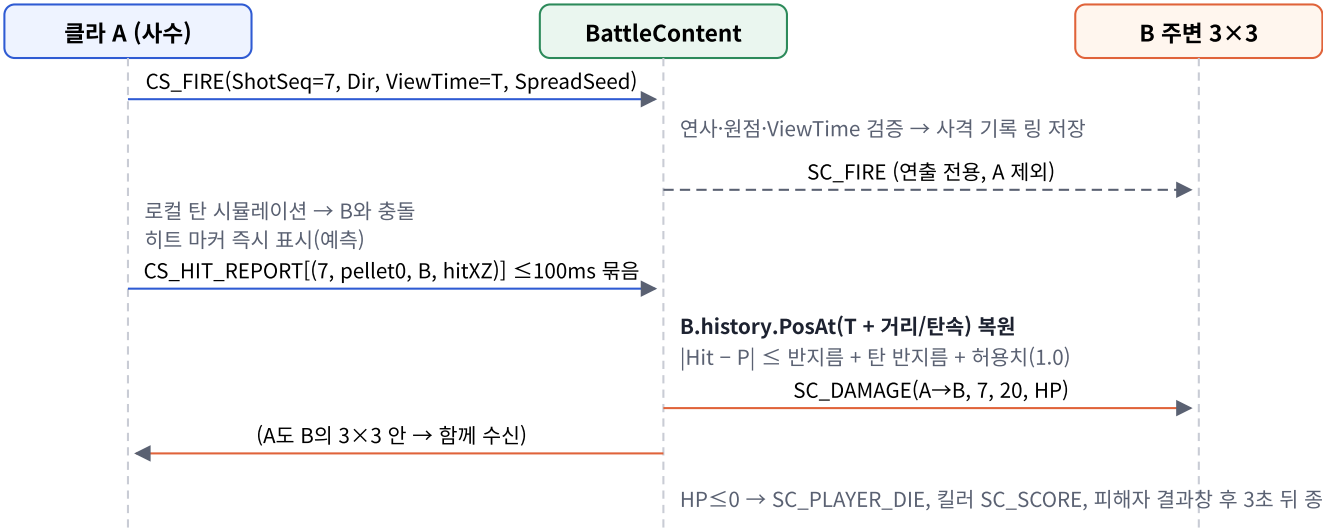


그림 8-1. 사격 → 피격 보고 → 되감기 검증 시퀀스

위치 이력 (PositionHistory)

- 플레이어마다 고정 링 64개 {t, x, z, vx, vz}. 스폰·통과한 이동·보정·구르기 시작/도착·정지 중 200ms 보충 시 기록 → 3초 이상 보관.
- PosAt(t): 두 기록 사이는 선형 보간, 최신 이후는 속도로 최대 200ms 외삽, 가장 오래된 기록 이전은 **판정 불가 → 거부**.

산탄 재현: 시드 공유

클라가 SpreadSeed 를 실어 보내고 서버가 SC_FIRE 로 그대로 전달한다. 산탄 각도는 32bit 해시 PelletRand(seed, i) 로 C#·C++이 비트 단위로 같게 계산 → 관찰자도 사수와 같은 산탄을 그린다. 서버는 각도를 재계산하지 않고 허용 범위로만 검증해 비용을 줄였다.

명중 항목 검증 9단계

1-3	사격 기록 존재 · 산탄 번호 범위 · 관통 수/중복 대상
4	대상 생존 · 본인 아님 · 서로 3×3 시야
5	보고 시간 ≤ 사거리/탄속 + 500ms + 200ms
6-7	거리 ≤ 사거리+탄반지름, 각도 ≤ 퍼짐/2 + 3°
8	과거 위치: ViewTime + 비행 시간 시점 위치와 비교
9	원점→명중점 선분이 벽·멸절한 장애물을 지나면 거부

한계를 인정하고 묶어 두기 사수 기준 판정이라 피해자는 "피했는데 맞는" 경험을 할 수 있다. ViewTime을 조작해도 되감기 상한 max_rewind_ms (500)를 넘을 수 없고, 미래 시각(-50ms 초과)은 거부한다. 이력 시각이 대상 패킷 도착 시각이라 생기는 편도 지연 오차는 hit_tolerance 로 흡수했다.

시각 동기화

되감기 검증이 성립하려면 클라가 "지금 화면에 보이는 적이 서버 시각으로 언제인지"를 알아야 한다.

```
rtt          = now - ClientTimeMs
offsetSample = ServerTimeMs + rtt/2 - now
offset       = 최근 8개 중 RTT 최소 샘플의 offset
              (초당 50ms 이내로 완만히, 1초 이상 차이는 즉시 스냅)
EstServerNow = localNow + offset
ViewTimeMs   = EstServerNow - InterpDelay(100ms)
```

- 입장 직후 200ms 간격 3회로 빠르게 수렴, 이후 2초마다 CS_PING.
- RTT 최소 샘플을 쓰는 이유: 큐잉 지연이 적은 샘플일수록 편도 = RTT/2 가정이 맞다.
- 좌상단 HUD에 Ping / avg / min, 색으로 품질 표시(F3 토글).
- **하트비트**: WebGL은 JS setInterval, 에디터는 스레드 타이머에서 60초마다 전송 → 탭이 백그라운드라 Unity 루프가 멈춰도 서버의 3분 타임아웃에 걸리지 않는다.

치트 대응 · 연결 종료 정책

검증 실패를 "위반"으로 세고 짧은 창 안에서 임계치를 넘으면 끊는다. 핵심은 **정상 플레이에서도 생기는 거부를 위반으로 세지 않는 것이다**.

단계	조건	처리
NetLib	Code≠119, Len>512, 틱당 수신 >2047B, 송신 큐 >1000, 빈 recv 10회	즉시 종료
입장	10초 내 미입장, 입장 전 다른 패킷	KICK 후 0.3초
게임	알 수 없는 타입·길이 불일치, 3분 무수신	KICK 후 0.3초
게임	이동 위반 5초 내 10회	CHEAT_SUSPECT
게임	부정 행위 10초 내 20회	CHEAT_SUSPECT

위반 분류 (ViolationKind)

이동: move:value / speed / blocked / budget, roll:cooldown
부정: fire:weapon / seq / dir / origin>3m / viewtime-future / rate, hit:no-shot / pellet / pierce / dup-target / range / angle / position / wall ...

세지 않는 거부 (ignored-hit)

대상이 이미 사망·퇴장, 시야 밖으로 나감, 보고 시한 초과, 대상 이력이 아직 없는 시각, 서버가 거부한 과거 사격의 명중. — 지연이 큰 정상 유저가 킥되던 문제에서 도출(15장 ①).
킥 시 게임 로그에 한 줄: 기간 내 횟수·마지막 종류·종류별 누적·ignored-hit 수·플레이 시간 → 오탐인지 사후 분석 가능.

맵 파이프라인 · 파괴 가능 업페물

레벨 디자인 도구는 그림판이다. 1500×1500 BMP 한 장(1픽셀 = 1m)이 서버 판정, 클라 메시, 충돌 격자, 전체 맵 텍스처의 단일 원본이 된다.

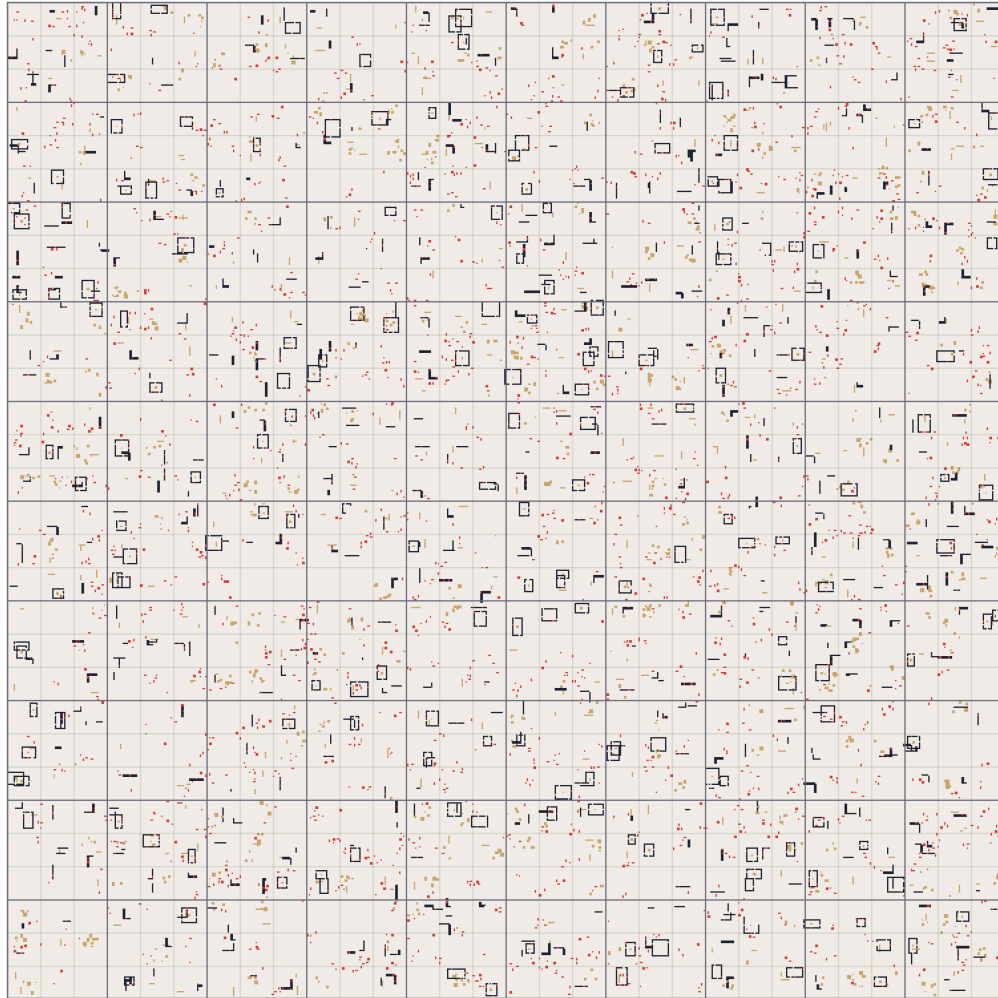


그림 11-1. 예시 맵 전체 (굵은 선 = 구역 150m, 얇은 선 = 섹터 50m)



그림 11-2. 중앙 300m 확대 — 검정 벽 · 베이지 낮은 업페물 · 빨강 파괴 가능

맵 규칙 · 엄폐물 동기화

색 → 칸 규칙

색	종류	이동	총알
$R \geq 150, G \cdot B \leq 100$	파괴 가능 (R로 체력 10~100)	막힘→파괴 후 통과	막힘→통과
밝기 < 64	벽	막힘	막힘
$64 \leq \text{밝기} < 224$	낮은 엄폐물	막힘	통과

같은 R로 상하좌우 연결된 칸 묶음 = 엄폐물 1개, id는 z·x 오름차순 스캔 순서(서버·클라 동일). 예시 맵: 벽 23,709칸 · 낮은 11,765칸 · 파괴 가능 10,339칸(3,000개), `generate_example_map.py` (numpy·scipy, 시드 고정).

맵 해시로 불일치 차단

칸 값을 FNV-1a 32bit로 해시해 서버는 `SC_ENTER_GAME.MapHash`로 보내고, 클라는 임포트 시 계산한 값과 비교해 다르면 경고, 더미는 `MAP_MISMATCH`를 남긴다. 맵이 어긋나면 정상 이동이 `move:blocked`로 키투기 때문에 생긴 안전장치다.

파괴 가능 엄폐물 상태 동기화

- 방마다 체력·파괴 시각 보유. 피격은 사수 보고 → 서버 검증(엄폐물 칸 0.75m 이내, 다른 엄폐물 관통 금지).
- 체력 변화는 주변 3×3 (`SC_COVER_HP`), 파괴·재생은 방 전체(`SC_COVER_STATE`, 입장 시 파괴 목록).
- 30초 뒤 재생하되 **그 칸에 플레이어가 겹쳐 있으면 비킬 때까지 대기** — 벽 속에 갇히는 상황 방지.
- 서버 이동 검증·클라 충돌이 방의 파괴 상태를 따름. 전환 순간의 어긋남은 보정만 하고 위반으로 세지 않는다.

게임 시스템

넓은 맵의 드문 교전이라는 설계 의도를 시스템으로 받쳐 준다: 스폰은 사람을 피하고, 에어드랍은 사람을 모은다.

스폰 — 가장 한산한 곳

900개 섹터마다 자신+인접 8섹터의 생존 인원을 세어 **최소인 섹터들 중 무작위** → 그 안의 무작위 위치(경계 4m 안쪽). 엄폐물과 겹치면 `FindFree` 로 가장 가까운 빈 칸. 고정 스폰 49개 방식(v0.7)에서 바꾼 것으로, 스폰 직후 교전을 줄였다. 테스트 모드는 지정 섹터에 몰아 스폰.

컨테이너 · 루팅

사망 지점에 가방(30초, 3×3 시야로만 보임), 에어드랍(방 전체). F를 1초/2초 누르면 서버가 거리 3m·"마지막 이동 후 경과 ≥ 필요 시간 - 250ms"를 검증해 내용 전송. 여러 명이 동시에 열 수 있고 같은 아이템은 **서버에 먼저 도착한 요청만 성공**, 내용이 바뀌면 열어 둔 모두에게 갱신.

점수 · 랭킹

킬 시 $+1 + \text{floor}(\text{피해자 점수} \times 0.5)$. 정렬: 점수 → 먼저 달성한 시각 → ID. TOP3의 (ID, 점수, 순서)가 바뀌면 그 틱 끝에 1회만 방송(같은 틱 내 여러 변경 병합).

에어드랍 — 가장 붐비는 곳

1분 주기, 방 인원 30명당 1개(최대 7). 투하 30초 전에 위치를 방 전체에 예고. 위치는 생존 인원이 가장 많은 3×3 묶음이되 기존·예정 지점과 체비셰프 거리 3 미만은 제외. 빈 방에 첫 입장 시 즉시 1개. 특수 무기를 가져가면 즉시 제거 → 쟁탈을 유도.

무기 · 인벤토리

슬롯 1 특수 무기(내구도), 2 권총(무한), 3 붕대(최대 5). 서버가 장착 무기를 알고 있어 다른 무기 ID의 사격은 거부. 사격 토큰 상한 `clamp(1+500/interval, 1, 3)` 으로 무기 전환 직후 느린 무기의 연속 발사를 막는다. 붕대 2초 사용 중 감속, 사격·구르기·전환 시 서버에서 취소.

사망 처리 순서

섹터 제거·상태 DEAD(이후 입력 무시) → 가방 생성 → 3×3에 `SC_PLAYER_DIE` → 킬러 점수 → 피해자에게 결과 송신 후 **3초 뒤** 연결 종료. 즉시 끊으면 결과 패킷이 송신 큐에서 유실될 수 있기 때문.

클라이언트 아키텍처

Unity WebGL은 소켓·스레드를 쓸 수 없다. 통신 계층을 플랫폼별 WebSocket 백엔드 뒤로 숨기고, 상위 코드는 바이트 스트림만 다루도록 했다.

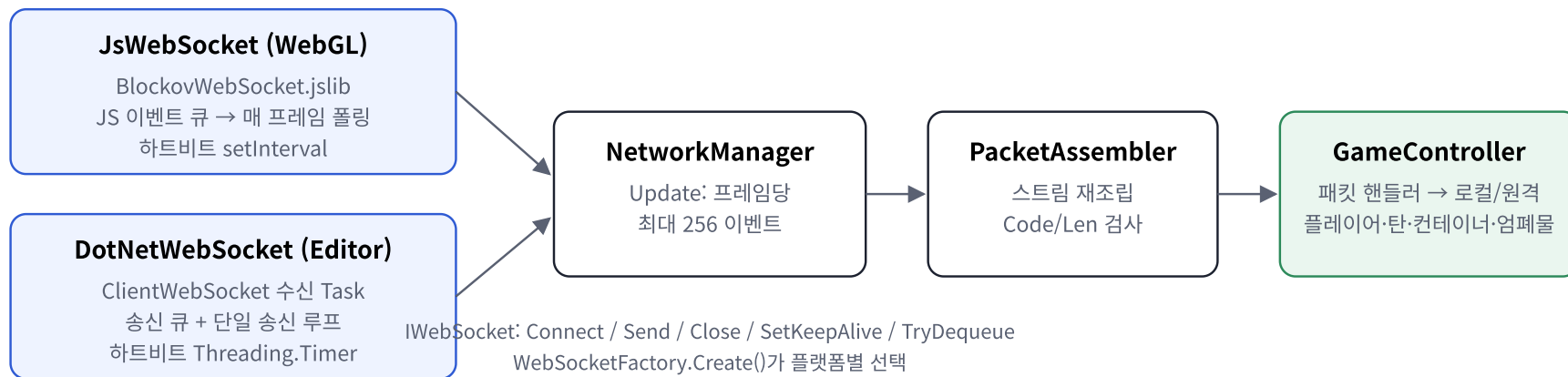


그림 13-1. 통신 모듈 — 백엔드 교체 가능, 상위 로직은 플랫폼 무관

주요 컴포넌트

LocalPlayerController	입력 즉시 이동(예측), 0.25m 충돌·축별 미끄러짐, 구르기, 조준선(벽에서 끊김), 흔들림·산탄 시드 사격
RemotePlayer	스냅샷 보간(~100ms), 외삽 최대 200ms
Projectile / HitReporter	로컬 탄만 충돌 판정, 관찰자 탄은 연출. 보고 100ms/29건 flush
ServerClock / LatencyHUD	시각 오프셋 추정, RTT 표시
ContainerManager / CoverManager	가방·에어드랍·예고, 엄페물 상자·체력·재생 게이지
WeaponRegistry	서버 수치 + 로컬 프리팹·사운드 결합

WebGL에서 부딪힌 것들

- **분홍색 오브젝트:** CreatePrimitive 기본 머티리얼이 URP WebGL 빌드에서 깨짐 → 공용 URP Lit 머티리얼 + MaterialPropertyBlock 색 지정(RuntimeMaterials.Apply)으로 통일.
- **한글 표시·입력:** IMGUI 기본 폰트에 한글 없음 → Noto Sans KR 서브셋 폰트, 줄 높이 맞춤으로 글자 잘림 해결. 이름 입력은 HTML <input> 오버레이(jslib)로 IME 지원.
- **백그라운드 탭:** Unity 루프가 멈춰도 JS 타이머로 하트비트 유지.
- **접속 주소:** 페이지를 준 호스트의 /ws 를 자동 사용(http→ws, https→wss), ?server= 로 덮어쓰기.
- **씬 전환 중 패킷:** 입장 응답 이후 패킷을 보류했다가 전투 씬 준비 후 순서대로 처리.

테스트 · 부하 측정 · 운영

로직 테스트, 실제 서버를 두드리는 C++ 더미 클라이언트, 운영 중 문제를 남기는 로그·덤프·대시보드.

① DummyClient (C++ IOCP, 최대 2만)

- 서버의 `GameProtocol.h·ObstacleMap.cpp` 를 그대로 컴파일 → 맵·프로토콜 정의가 한 곳.
- **교전 AI**: 시야선이 확보된 가장 가까운 적 주위를 옆걸음으로 돌며 사격, 거리 유지, 구르기 회피, 에어드랍 루팅 후 특수 무기 사용.
- 약함 $w \in [\min, \max]$ 하나로 반응 지연·사격 간격·조준 오차·리드 사격 확률을 연동 → 설정 3줄로 난이도 조절.
- 명중 보고는 실제 탄 직선이 대상 예상 위치를 지날 때만 → **서버 검증의 오탐을 재는 도구**로도 쓴다.
- 무인 모드 `--count N --duration s`: 5초마다 통계 한 줄.

② 측정 결과 (로컬 PC 1대)

5,000

동시 게임 중(이동) · 서버 송신 195k pkt/s · CPU 23%

48ms

위 부하에서 평균 RTT

~1,500/s

3,000명 사격 시 서버 확인 명중

0 · 0

300명 48시간 교전: 위치 보정 · 킥

대규모 설정은 `test/stress/game_config.txt`.

측정 환경(서버·더미 동일 PC): Intel Core i5-13600KF · 메모리 32GB · Realtek Gaming 2.5GbE Family Controller

③ 도구

- **콘솔 대시보드**(1초 갱신, 스크롤 없음): 세션·방별 인원, 송수신 KB/s·pkt/s, 패킷 풀, CPU, 메모리 ·NonPaged pool, NIC, 맵 로드 상태·해시, 최근 로그 10줄.
- **게임 로그 파일**: 방 시작, 입장 거부, 무기표 경고, 에어드랍, 킥 상세, 크래시. 다른 프로그램이 읽는 중에도 기록.
- **크래시 덤프**: 서버·더미 공용 `CrashDump.h`, 같은 빌드 .pdb로 VS에서 분석.
- **원클릭 실행**: `start_server.bat` 이 서버·게이트웨이·웹서버를 각 창으로, `build` 면 vswhere로 MSBuild를 찾아 빌드.
- **외부 공개**: 포트포워딩 80 하나. 8080·10301은 루프백 전용, 게이트웨이에서 Origin·IP당 연결 수 제한.

트러블슈팅 사례

서버 쪽에서 원인을 추적해 구조로 막은 세 가지 사례.

① 지연이 큰 정상 유저가 치트 의심으로 킁됨

증상	교전이 잦거나 RTT가 높은 환경에서 정상 플레이어가 CHEAT_SUSPECT 로 끊겼다(부정 행위 10초 내 20회 규칙).
원인	명중 검증의 거부를 사유와 무관하게 모두 부정 카운트에 넣고 있었다. 사수 판정·서버 검증 구조에서는 지연만으로 거부가 생긴다 — 보고 도착 전에 대상이 죽음·퇴장, 그 사이 3×3 시야 밖으로 이동, 보고 시한(사거리/탄속 + 되감기 상한) 초과, 대상 위치 이력에 아직 없는 시각. 조작의 신호가 아니라 네트워크의 성질이다.
해결	거부 사유를 둘로 나눠 정상 플레이에서도 생기는 것은 ignored-hit 로만 집계하고, "보낸 적 없는 미래 ShotSeq에 대한 명중"처럼 조작으로만 생길 수 있는 경우만 카운트. 킁 시 게임 로그에 기간 내 횟수·마지막 종류·종류별 누적·ignored-hit 수·플레이 시간을 한 줄로 남겨 사후 판별. Node 봇 테스트에 "늦은 보고는 킁 없음 / 미래 seq는 킁" 시나리오 추가.
결과	더미 300명 48시간 교전 테스트에서 위치 보장 0회 · 킁 0회.
배운 점	검증 로직은 막는 것보다 "정상 유저를 막지 않는 것"을 먼저 증명해야 한다. 오탐 0을 측정 가능한 목표로 두고 더미 클라이언트로 확인하는 절차를 만들었다.

② 맵 데이터 불일치로 정상 이동이 거부됨

증상	클라·더미는 벽 옆을 정상적으로 지나는데 서버가 move:blocked 로 위치를 계속 보정하다 이동 위반(5초 내 10회)으로 킁.
원인	서버 설정의 맵 경로가 잘못돼 서버만 엄폐물 없이 실행되고 있었다. 이동 규칙은 같고 입력 데이터가 달랐던 것. 서버는 맵을 못 읽어도 경고만 내고 계속 실행되므로 문제가 판정 오류의 모습으로 나타나 원인을 찾기 어려웠다.
해결	맵 해시: 1500×1500 칸 값을 FNV-1a 32bit로 해시해 SC_ENTER_GAME.MapHash 로 전달 → 클라는 임포트 시 해시와 비교해 화면 경고, 더미는 MAP_MISMATCH 로그, 서버 대시보드 [Map] 줄에 로드 상태·칸 수·해시 상시 표시. 경로 문제 제거: 작업 폴더에 game_config.txt 가 없으면 실행 파일 위쪽 폴더를 찾아 작업 폴더를 옮김 → exe를 직접 실행해도 상대 경로가 맞는다(더미도 동일).
배운 점	여러 구현이 같은 데이터로 판정하는 구조에서는 데이터가 같다는 것 자체를 검증해야 한다. 프로토콜 버전 검사와 맵 해시는 같은 원리 — 어긋남을 조용한 버그가 아니라 즉시 보이는 경고로 바꾸는 장치다.

③ 패킷 상한 확장 중 발견한 수신 버퍼 오버플로와 헤더 미초기화

배경	게임 프로토콜에 맞추기 위해 NetLib의 패킷 페이로드 상한을 127B에서 512B로 늘렸다.
원인	수신 스택 버퍼 크기 오류 : 위커가 패킷을 꺼낼 때 쓰는 스택 버퍼(<code>iobuf / serializeBuf</code>)가 헤더 5B를 빼고 잡혀 있어 최대 크기 패킷이 오면 헤더만큼 스택을 넘어 쓴다. 헤더 미초기화 : 브라우저 클라 때문에 암호화를 끄고(<code>nullptr</code>) 쓰는데, 이때 송신 헤더의 <code>Code·RandKey·Checksum</code> 이 초기화되지 않은 채 나갔다. 수신측은 <code>Code</code> 를 항상 검사하므로 쓰레기 값이면 연결이 끊긴다.
해결	수신 스택 버퍼를 <code>PAYLOAD_LEN_DEFAULT + 헤더 크기</code> 로 자동 계산. 소켓 <code>recv</code> 링버퍼를 128B → 2048B로 늘리고 상수화(<code>SESSION_RECV_BUFFER_SIZE</code>). 암호화 off 시 헤더를 <code>Code 0x77, RandKey·Checksum 0</code> 으로 명시적으로 채움. 상한 변경 시 함께 고칠 상수 목록(<code>Packet.h, SimpleEncoder.h, NetLibDefine.h</code>)을 라이브러리 문서에 명시. 직렬화 버퍼가 넘치는 쓰기를 조용히 무시하므로 게임 서버의 모든 목록 패킷은 원소 크기로 상한을 계산해 분할 송신(<code>CREATE</code> 9명, <code>COVER_STATE</code> 56개 단위)하도록 규칙화.
배운 점	상수 하나를 바꿀 때는 그 값에서 파생된 크기가 곳곳에 하드코딩돼 있는지 의심해야 한다. 크기를 하나의 원본 상수에서 계산되게 바꿨다.

확장 계획 · 회고

명세서의 열린 이슈와 라이브러리의 알려진 이슈를 다음 단계 과제로 정리하고, 개발 과정을 돌아본다.

확장 계획		
#	과제	내용
1	NetLib 네이티브 WebSocket	게이트웨이 홉 제거. 세션 상태 HANDSHAKE → OPEN , HTTP Upgrade 응답(SHA-1·Base64), WS 프레임 파서를 NetHeader 파서 앞단에. 송신은 여러 패킷을 WS 메시지 1개로 묶고 멀티캐스트는 WS 헤더 공유. TLS는 앞단 종단 + PROXY protocol로 실제 IP 수신. "WS 메시지 ↔ 패킷 계약"을 유지하므로 클라·컨텐츠 코드는 변경 없음
2	밀집 트래픽 절감	수신자별 이동 묶음 SC_MOVE_BATCH , 위치 16bit 양자화, 원거리 대상 갱신 주기 낮추기 (밀집 비용이 시야 내 인원의 제곱)
3	Content 스케줄링	바쁜 루프라 유틸 시에도 방 4개 CPU 약 24% → 타이머 기반 대기로 유틸 비용 제거
4	시야 기반 정보 필터링	엄폐물이 정보 송신을 막지 않아 벽 뒤 적 정보가 클라에 존재 → 시야선 기반 필터링으로 월핵 여지 축소
5	설정 단일화	구르기·봉대·상호작용 수치를 클라 상수가 아닌 서버 설정에서 수신 (무기 수치는 이미 그렇게 함)
6	통계 기반 치트 탐지	사수 판정 구조에서 정교한 에임봇은 개별 검증만으로 막기 어려움 → 명중률·반응 시간 분포로 이상치 탐지

회고

잘한 것

- 명세서를 "현재 동작" 문서로 유지했다. 패킷 바이트 크기·검증 수식·설정 키까지 적어 서버·클라·더미·테스트 봇 네 구현이 같은 기준을 공유했고, 변경 이력이 그대로 개발 일지가 됐다.
- 판정 데이터를 한 원본에서 파생(맵 BMP, 무기표, 프로토콜 헤더)시키고 해시·버전으로 대조했다(사례 ②).
- 기준에 만든 NetLib의 구조를 게임에 그대로 맞췄다. 방 = Content로 두어 "방 = 단일 스레드 = 락 없는 게임 상태"를 얻었고, 방 사이 공유는 불변 데이터와 원자 카운터뿐이다.
- 검증의 기준을 "오탐 0"으로 잡고 측정했다. 이를 위해 C++ 더미 클라이언트에 교전 AI와 강도 조절을 넣었다(사례 ①).

아쉬운 것

- 성능 측정이 로컬 PC 1대 기준이라 실제 네트워크의 손실·지터 환경 수치가 없다.
- 게이트웨이를 거치는 동안 서버는 모든 접속을 게이트웨이 IP로 본다 — IP 기반 방어가 서버 밖에 있다.
- 그래픽·UI가 임시(캡슐, ImGui) 상태라 게임으로서의 완성도는 낮다.

AI 협업

서버 엔진(NetLib·Utils)은 직접 만든 것을 그대로 쓰고, 그 위의 게임 콘텐츠·클라이언트·도구 구현에 AI(Claude)를 사용했다(01 · 개발 분담). 나는 기획·요구 사항·설계 판단과 결과 검증(테스트 봇·더미 부하 테스트)을 맡았다. 명세를 구체적으로 쓸수록 구현이 한 번에 맞았고, 이것이 명세서를 바이트 단위까지 유지한 이유이기도 하다.

저장소 구성 server/ContentEchoServer (NetLib·Utils) · server/GameServer · server/DummyClient · server/gateway · client/escape_from_blockov · map/ · start_server.bat · 플레이 choseongchan.kro.kr · GitHub github.com/ChokeOutCho/escape_from_blockov

escape_from_blockov · Game Portfolio

감사합니다

조성찬

https://github.com/ChokeOutCho/escape_from_blockov

[플레이 영상 \(YouTube\)](#)